



SQL Básico

El diseño de una base de datos es un proceso clave que requiere planificación y análisis. Antes de comenzar a crear tablas, es fundamental definir qué información vamos a almacenar. En el modelo relacional, el esquema de la base de datos debe ser lo más estable posible a lo largo del tiempo, por lo que es recomendable dedicar el tiempo necesario para diseñarlo correctamente.

Para definir un modelo de datos relacional, primero debemos identificar qué elementos del mundo real queremos representar dentro de la base de datos. Este proceso se basa en tres aspectos fundamentales:

- **Identificación de entidades:** Determinar cuáles son los objetos del mundo real que deseamos representar.
- **Definición de atributos:** Establecer qué características o propiedades describen cada entidad.
- **Relaciones entre entidades:** Identificar cómo interactúan y se vinculan entre sí.

Las entidades se representan en forma de tablas, donde cada columna corresponde a un atributo y cada fila representa un registro único de esa entidad. Por ejemplo, en una base de datos de automóviles, la entidad “Auto” podría tener atributos como número de serie, marca, submarca, modelo, color y tipo de transmisión. Observa el siguiente ejemplo:

Número de serie	Marca	Submarca	Modelo	Color	Transmisión	Recorrido
3VWU67FR7RE3	Volkswagen	Polo	2015	Gris Platino	Manual	125,000
HDG6JBSD7FDT	Honda	Civic	2020	Plata Diamante	Automática	40,000
3VWGCISDS7D7	VW	Tiguan	2012	Blanco	Automática	132,000
6TDG675E54GY	Toyota	Corolla	2022	Rojo	Manual	30,000

Un concepto esencial en bases de datos es el de **llave primaria**. Se trata de un atributo único e irrepetible que permite identificar de manera exclusiva cada registro en una tabla.

Por ejemplo, en la tabla de automóviles, el número de serie puede ser la llave primaria, ya que no es posible que existan dos autos con el mismo número de serie. En otros contextos, como una base de datos de estudiantes, la matrícula puede funcionar como llave primaria, pues es un valor único asignado a cada alumno. De manera similar, en una empresa, cada trabajador tiene un número de empleado que lo distingue de los demás.

Reglas clave de una llave primaria:

- No puede haber valores repetidos.
- No puede contener valores nulos (NULL).

Para modelar las relaciones entre entidades, se emplean las **llaves foráneas**. Una llave foránea es un atributo en una tabla que hace referencia a la llave primaria de otra tabla.

Por ejemplo, si queremos almacenar información sobre marcas y modelos de autos en tablas separadas, podríamos estructurar la base de datos de la siguiente manera:

- Una **tabla de marcas** con una llave primaria "ID_Marca".
- Una **tabla de autos**, donde el campo "ID_Marca" actúe como llave foránea, vinculando cada auto a su respectiva marca.

Esto ayuda a evitar errores e inconsistencias en los datos. Más adelante hablaremos sobre el concepto de **normalización**, una técnica que mejora el diseño de nuestra base de datos. La normalización se basa en un conjunto de reglas que ayudan a hacer la base de datos más robusta y a evitar inconsistencias.

Por ejemplo, si trabajamos con automóviles, no es recomendable almacenar la marca del vehículo tal como la escriben los usuarios, ya que pueden hacerlo de diferentes maneras. Tomemos como ejemplo la marca Volkswagen: algunas personas pueden escribirla como "Volkswagen", "VOLKSWAGEN", "VW", entre otras variaciones. Esto puede generar problemas en la gestión de los datos, por lo que es importante aplicar técnicas de normalización para garantizar la uniformidad en la información almacenada.

Para evitar múltiples formas de almacenar un mismo concepto, podemos crear tablas específicas que nos ayuden a gestionar estas entidades. De esta manera, en lugar de permitir que el usuario escriba libremente el valor de un atributo, le ofrecemos una selección de opciones predefinidas.

Por ejemplo, si creamos una tabla de marcas de automóviles, podríamos establecer relaciones entre marcas y submarcas. Así, podríamos indicar que el modelo Civic pertenece a la marca Honda o que la submarca Corolla corresponde a la marca Toyota.

Es fundamental recordar que, al trabajar con llaves foráneas, estas deben contener un valor existente en la llave primaria de la tabla a la que hacen referencia o, en su defecto, un valor nulo.

Las llaves foráneas pueden incluir restricciones conocidas como **restricciones de integridad referencial**, que determinan el comportamiento de la base de datos cuando se actualiza o elimina una llave primaria referenciada.

Al declarar una llave foránea, podemos definir cómo debe reaccionar la base de datos ante la eliminación o actualización de la tupla referenciada. Las opciones más comunes son:

- **Restringir:** Si elegimos restringir, la base de datos impedirá la eliminación de una fila si existen registros que la referencien. Por ejemplo, si intentamos eliminar la marca Volkswagen de la tabla de marcas, pero hay vehículos registrados con esta marca, la operación será rechazada hasta que eliminemos primero todos los vehículos de Volkswagen.
- **Asignar valor nulo:** Si optamos por asignar `NULL`, al eliminar la marca Volkswagen de la tabla de marcas, todos los registros de vehículos que hacían referencia a esta marca se actualizarán automáticamente con un valor nulo en el campo correspondiente.
- **Propagar:** Si seleccionamos la opción de propagación, cuando eliminemos una marca, todos los vehículos asociados a ella se eliminarán automáticamente.

Estas opciones también aplican cuando se actualiza una llave primaria, tanto para el borrado como para la actualización. Podemos elegir que:

- **La actualización esté restringida** y no se permita modificar la llave primaria si existen registros que la referencien.
- **La actualización se propague en cascada**, de modo que cualquier cambio en la llave primaria se refleje automáticamente en todas las referencias.
- **No se permita la actualización**, asegurando que la base de datos rechace cualquier intento de modificar la llave primaria mientras existan registros que dependan de ella.

Al definir estas restricciones, garantizamos la consistencia e integridad de nuestra base de datos.

Creación de bases de datos y tablas (CREATE DATABASE, CREATE TABLE)

SQL es un lenguaje diseñado para gestionar bases de datos que utilizan el modelo relacional, en el cual la información se organiza en tablas. Estas tablas pueden visualizarse como arreglos bidimensionales compuestos por filas y columnas.

- Cada fila representa un registro o una tupla.
- Cada columna corresponde a un atributo de la tabla.

Una **tupla** es un conjunto de valores que corresponden a los atributos de una tabla. Cada atributo debe cumplir con un dominio de datos, lo que significa que tiene un tipo de dato específico asignado.

Por ejemplo, en una tabla de automóviles:

- La marca del auto puede almacenarse como una cadena de texto (*string*) con una longitud máxima de 30 caracteres.
- La submarca también se puede representar como un *string* de hasta 30 caracteres.
- El modelo del auto puede estar representado por un número de cuatro dígitos, como 2024 o 2025.

Aquí es donde entra el concepto de **tipos de datos** que podemos trabajar en SQL.

Tipos de datos en MySQL

SQL ofrece una gran variedad de tipos de datos, que pueden variar según el sistema gestor de bases de datos utilizado. Por ejemplo, MySQL puede introducir o eliminar tipos de datos en diferentes versiones. Para obtener información actualizada, siempre es recomendable consultar la documentación oficial del desarrollador.

Nota: En la sección de Material Adicional, al finalizar esta lectura, encontrarás un enlace donde podrás revisar en detalle los tipos de datos disponibles en MySQL. Recuerda que, a lo largo de este tema, trabajaremos con MySQL debido a que es un sistema de gestión de bases de datos ampliamente utilizado y compatible con Windows, Mac y Linux.

Los **tipos de datos** en MySQL son similares a los de otros sistemas gestores de bases de datos, lo que facilita la compatibilidad entre ellos. En MySQL, podemos clasificar los tipos de datos en tres categorías principales:

- 1 **Tipos de datos numéricos**
- 2 **Tipos de datos de texto**
- 3 **Tipos de datos de fecha y hora**

A continuación, conocerás más sobre cada tipo de datos.

Tipos de datos numéricos

Los tipos de datos numéricos en MySQL incluyen valores enteros y decimales.

Cuando trabajamos con **valores enteros**, es común pensar en el tipo de dato `INT`, ya que es ampliamente utilizado en muchos lenguajes de programación como C, C++ y Java. Sin embargo, MySQL ofrece varios tipos de datos enteros que permiten optimizar el almacenamiento según el rango de valores que necesitamos. Observa la siguiente tabla:

Tipo de dato	Rango con signo	Rango sin signo
<code>TINYINT</code>	-128 a 127	0 a 255
<code>SMALLINT</code>	-32,768 a 32,767	0 a 65,535
<code>MEDIUMINT</code>	-8,388,608 a 8,388,607	0 a 16,777,215
<code>INT</code> o <code>INTEGER</code>	-2,147,483,648 a 2,147,483,647	0 a 4,294,967,295
<code>BIGINT</code>	-9 cuatrillones a 9 cuatrillones	0 a 18 cuatrillones

Como puedes ver, `TINYINT` es el entero más pequeño, que puede guardar valores desde -128 a 127 (o de 0 a 255 sin signo), ideal para datos con valores limitados. `SMALLINT` es un entero de menor tamaño que guarda valores de -32,768 a 32,767, mientras que `MEDIUMINT` es un entero mediano que maneja valores entre -8,388,608 y 8,388,607. `INT` o `INTEGER` es el tipo entero más común, con un rango de -2,147,483,648 a 2,147,483,647. Finalmente, `BIGINT` es un entero grande que permite almacenar valores desde -9 cuatrillones hasta 9 cuatrillones, útil para datos de gran magnitud.

Otro tipo de dato numérico muy útil es `DECIMAL`, que permite definir con precisión cuántos dígitos se utilizarán en la parte entera y cuántos en la parte decimal de un número.

Por otro lado, existen los tipos de datos `FLOAT` y `DOUBLE`, con los que probablemente ya estés familiarizado si has trabajado con lenguajes de programación. Ambos representan números con punto decimal, pero con distinta precisión: `FLOAT` tiene precisión simple, mientras que `DOUBLE` ofrece el doble de precisión, permitiendo valores más exactos en cálculos numéricos.

■ Tipos de datos de texto

Hablemos ahora de los tipos de datos de tipo texto. En MySQL, este tipo de datos permite almacenar cadenas de caracteres de distintas longitudes. Los más básicos son:

Tipo de dato	Descripción
<code>CHAR (N)</code>	Cadena de caracteres de longitud fija. Se usa cuando la cantidad de caracteres siempre es la misma, como una matrícula o un número de empleado (entre 8 y 12 caracteres, por ejemplo).
<code>VARCHAR (N)</code>	Se utiliza para almacenar una cadena de caracteres de longitud variable. Debemos pasarle un parámetro, es decir, indicar cuál es el tamaño máximo de esa cadena de caracteres de longitud variable.
<code>TEXT</code>	Se usa para almacenar grandes volúmenes de texto. Tiene variantes según el número de caracteres a almacenar, como el resumen de un libro o la semblanza de una persona: <code>TINYTEXT</code> , <code>TEXT</code> , <code>MEDIUMTEXT</code> y <code>LONGTEXT</code> .

Tipos de datos de fecha y hora

MySQL ofrece varios tipos de datos diseñados para manejar fechas y tiempos con precisión. Observa la siguiente tabla:

Tipo de dato	Descripción	Ejemplo
DATE	Almacena solo la fecha en formato YYYY-MM-DD .	2025-03-14
DATETIME	Almacena fecha y hora en formato YYYY-MM-DD HH:MM:SS . Se usa en registros de eventos o actividades.	2025-03-14 15:30:00
TIMESTAMP	Similar a DATETIME , pero también almacena la zona horaria. Es útil para aplicaciones globales.	2025-03-14 15:30:00 UTC
TIME	Guarda solo la hora en formato HH:MM:SS .	13:22:59
YEAR	Almacena solo el año en formato de cuatro dígitos. Útil para registros como el modelo de un auto.	2025

Estos tipos de datos permiten optimizar el almacenamiento y mejorar la eficiencia en la manipulación de datos en MySQL.

Ahora que hemos revisado los tipos de datos disponibles en nuestro gestor de bases de datos, es momento de conocer cómo crear y manejar tablas en SQL. A continuación, crearemos un esquema de nuestra base de datos.

Creación y manejo de esquemas y tablas en MySQL

Recordemos que el esquema define la estructura en la que organizaremos la información, mientras que la base de datos contiene los datos que almacenaremos. Para crear una base de datos en MySQL, utilizamos el comando `CREATE DATABASE` seguido del nombre de la base de datos. Para este ejemplo, utilizaremos:

```
CREATE DATABASE agencia;
```

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.41-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE agencia;
Query OK, 1 row affected (0.01 sec)

mysql> 
```

Recuerda que cada instrucción en MySQL debe finalizar con un punto y coma (;).

Nota: MySQL permite la creación de múltiples bases de datos sin un límite específico, pero las restricciones pueden venir del sistema de archivos del sistema operativo.

Una vez que hemos creado las bases de datos necesarias, debemos indicarle al gestor de bases de datos cuál queremos utilizar. Para ello, usamos el comando `USE` seguido del nombre de la base de datos, como `USE autos`, `USE tienda` o `USE escuela`. Para este ejemplo, utilizaremos:

```
USE agencia;
```

```
mysql> USE agencia;  
Database changed  
mysql> █
```

Una vez que hemos seleccionado la base de datos con la que queremos trabajar, podemos comenzar a crear nuestras tablas.

Para conocer todos los detalles y opciones sobre la creación de tablas en MySQL, te recomendamos revisar la sección de Material adicional, donde encontrarás un enlace a la documentación oficial de MySQL.

Las tablas organizan los datos dentro de una base de datos. La sintaxis básica para la creación de una tabla es:

```
CREATE TABLE nombre_de_la_tabla (  
    lista_de_atributos_y_tipos  
);
```

La **lista de atributos** lleva primero el nombre del atributo, seguido del tipo de dato. De manera opcional, podemos especificar algunos detalles del atributo, como:

- **NOT NULL**: si queremos que el atributo no acepte valores nulos.
- **PRIMARY KEY**: para definir que el atributo será usado como llave primaria.

Además, si estamos usando un tipo de dato entero, podemos utilizar `AUTO_increment` para especificar que queremos que un campo específico almacene valores numéricos con un valor que automáticamente se incrementa de uno en uno.

■ Ejemplo 1. Creación de una tabla simple

Observa la siguiente imagen:

```
mysql> CREATE TABLE empleado(id SMALLINT NOT NULL, nombre VARCHAR(30) );
Query OK, 0 rows affected (0.06 sec)

mysql> █
```

En este caso, la tabla `empleado` contiene dos atributos:

- **id**: Un `SMALLINT` que no acepta valores nulos; sin embargo, no especificamos que este atributo sea una llave primaria.
- **nombre**: Un `VARCHAR(30)` que almacena una cadena de caracteres de longitud variable (máximo 30).

■ Ejemplo 2. Definiendo una llave primaria

Recreemos la misma tabla, pero ahora definiendo el atributo `id` como una llave primaria. De esta forma, la tabla NO aceptará dos tuplas con el mismo `id`, y tendremos un atributo para identificar de manera única a cada empleado.

Si queremos crear una tabla con el mismo nombre, como `empleado`, pero ya existe una con ese nombre en la misma base de datos, primero debemos eliminarla. Para hacerlo, usamos el comando `DROP TABLE` seguido del nombre de la tabla:

```
DROP TABLE empleado;
```

```
mysql> DROP TABLE empleado;
Query OK, 0 rows affected (0.04 sec)

mysql> █
```

Por otro lado, si lo que queremos es eliminar toda la base de datos, la instrucción sería: `DROP DATABASE nombre_de_base_de_datos`. Por ejemplo:

```
DROP DATABASE agencia;
```

```
mysql> CREATE TABLE empleado(id SMALLINT NOT NULL PRIMARY KEY, nombre VARCHAR(30) );
Query OK, 0 rows affected (0.05 sec)

mysql> █
```

Como puedes notar, solo agregamos `PRIMARY KEY` en la definición del atributo `id`.

Si necesitamos modificar un atributo, cambiar su tipo de dato o agregar un nuevo atributo a una tabla, podemos usar el comando `ALTER TABLE`. Para consultar detalles de cómo utilizar esta instrucción, puedes revisar la documentación oficial de MySQL.

■ Uso de llaves foráneas

Veamos un ejemplo de cómo definir llaves foráneas que hagan referencia a llaves primarias en otra tabla. En este caso, trabajaremos con una versión inicial de las tablas para nuestro ejemplo de autos.

- Tabla `marca`: Contendrá un `id` y el nombre de la marca.
- Tabla `auto`: Incluirá una llave foránea que hará referencia a la tabla `marca`.

La definición de las tablas es la siguiente:

```
CREATE TABLE marca(  
    id_marca SMALLINT NOT NULL,  
    nombre_marca VARCHAR(30) NOT NULL,  
    PRIMARY KEY(id_marca)  
);  
  
CREATE TABLE auto (  
    vin VARCHAR(17) ,  
    id_marca SMALLINT,  
    submarca VARCHAR(30) ,  
    modelo YEAR,  
    color VARCHAR(30) ,  
    transmission ENUM("MANUAL", "AUTOMATICA") ,  
    recorrido int,  
    FOREIGN KEY(id_marca) REFERENCES marca(id_marca)  
        ON DELETE RESTRICT  
        ON UPDATE CASCADE  
);
```

Aquí hay varias cosas importantes:

- No es necesario escribir toda la definición de la tabla en una sola línea. Podemos separarla en varias líneas para mejorar la legibilidad.
- En la tabla `marca`, declaramos la llave primaria de una manera diferente: en lugar de definirla junto al atributo, la especificamos al final de la lista de atributos con `PRIMARY KEY(id_marca)`. Ambas formas son válidas.
- En la tabla `auto`, utilizamos el tipo de dato `ENUM`, que permite definir un conjunto de valores de texto predefinidos para un atributo.
- En la tabla `auto`, al final de la lista de atributos, especificamos que la llave foránea será `id_marca` (un atributo de esta misma tabla). Usamos la palabra clave `REFERENCES` para indicar que esta llave foránea hace referencia al atributo `id_marca` de la tabla `marca`.

Finalmente, estamos especificando cómo queremos que se comporte la base de datos cuando queremos actualizar o eliminar una tupla referenciada.

```
mysql> CREATE TABLE marca(  
  ->   id_marca SMALLINT NOT NULL,  
  ->   nombre_marca VARCHAR(30) NOT NULL,  
  ->   PRIMARY KEY(id_marca)  
  -> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql> CREATE TABLE auto (  
  ->   vin VARCHAR(17) ,  
  ->   id_marca SMALLINT,  
  ->   submarca VARCHAR(30),  
  ->   modelo YEAR,  
  ->   color VARCHAR(30),  
  ->   transmission ENUM("MANUAL","AUTOMATICA"),  
  ->   recorrido int,  
  ->   FOREIGN KEY(id_marca) REFERENCES marca(id_marca)  
  ->     ON DELETE RESTRICT  
  ->     ON UPDATE CASCADE  
  -> );  
Query OK, 0 rows affected (0.08 sec)
```

¡Buen trabajo! Has aprendido cómo crear y manejar esquemas y tablas en MySQL, incluyendo la definición de llaves primarias y foráneas. Con estos conocimientos, podrás estructurar y organizar tus datos de manera eficiente en una base de datos relacional.

Insertión, eliminación y modificación de datos (INSERT INTO, DELETE FROM, UPDATE)

Una vez que hemos creado nuestra base de datos en MySQL, el siguiente paso es aprender a gestionar la información dentro de ella. Esto implica no solo insertar nuevos registros, sino también recuperarlos, modificarlos y eliminarlos según sea necesario.

Estas operaciones son fundamentales para manipular los datos de manera eficiente y garantizar que la información almacenada sea precisa y relevante. En esta sección, exploraremos cómo llevar a cabo cada una de estas acciones utilizando SQL.

Insertar datos

Ahora que hemos creado la estructura para guardar nuestros datos, vamos a ver cómo podemos insertar información. Utilizaremos el comando `INSERT INTO`, cuya sintaxis es:

```
INSERT INTO nombre_de_la_tabla (atributo1, atributo2, ...)
VALUES (valor1, valor2, ...);
```

Veamos algunos ejemplos a continuación.

■ Insertar un solo registro en la tabla `empleado`

Como primer ejemplo, vamos a insertar valores en la tabla `empleado`. Si no recuerdas su estructura, puedes usar el siguiente comando para consultarla:

```
DESCRIBE empleado;
```

```
mysql> describe empleado;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | smallint      | NO   | PRI | NULL    |       |
| nombre | varchar(30)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

En la descripción de la tabla `empleado`, podemos ver que tiene solo dos atributos: `id` y `nombre`.

Para insertar un único registro, usamos la instrucción `INSERT INTO` de la siguiente manera:

```
INSERT INTO empleado VALUES (1, "Luis Lopez");
```

```
mysql> INSERT INTO empleado VALUES (1, "Luis Lopez");
Query OK, 1 row affected (0.03 sec)
```

En este caso, la tupla que aparece después de la palabra `VALUES` tiene dos valores: uno para el campo `id` y otro para el campo `nombre`. En este ejemplo, como el nombre es un `varchar`, debe ir entre comillas: `"Luis López"`.

■ Insertar múltiples registros a la vez

También es posible insertar varios registros en una sola instrucción:

```
INSERT INTO empleado VALUES  
  (2, "Ana Sanchez"),  
  (3, "Raul Trejo"),  
  (4, "Jorge Campos") ;
```

```
mysql> INSERT INTO empleado VALUES  
->  (2, "Ana Sanchez"),  
->  (3, "Raul Trejo"),  
->  (4, "Jorge Campos") ;  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

■ Insertar valores en atributos específicos

Otra manera de insertar valores en una tabla es especificando que se quieren almacenar valores para algunos atributos. En el caso de nuestra tabla, insertaremos solo valores en el campo `id`. Esto es posible porque, al momento de crear la tabla, NO especificamos que el nombre no podía tener un valor nulo, por eso nos permite insertar registros solo con valor para el `id`:

```
INSERT INTO empleado (id) VALUES (5), (6);
```

```
mysql> INSERT INTO empleado (id) VALUES (5),(6);  
Query OK, 2 rows affected (0.01 sec)  
Records: 2  Duplicates: 0  Warnings: 0
```

Hasta ahora, solo hemos insertado valores en nuestra tabla, pero seguro te estás preguntando: ¿cómo puedo ver el contenido de la tabla para ver si, efectivamente, se han insertado los valores? Veamos a continuación la instrucción `SELECT`.

Consultar datos

Para poder recuperar o leer los datos en una tabla, utilizamos el comando `SELECT`, cuya sintaxis es:

```
SELECT lista_de_atributos FROM nombre_de_la_tabla;
```

Si queremos visualizar solo un atributo, por ejemplo, `nombre`, usamos:

```
SELECT nombre FROM empleado;
```

```
mysql> SELECT nombre FROM empleado;
+-----+
| nombre |
+-----+
| Luis Lopez |
| Ana Sanchez |
| Raul Trejo |
| Jorge Campos |
| NULL |
| NULL |
+-----+
6 rows in set (0.00 sec)
```

Si queremos ver todos los atributos, usamos el caracter `*`:

```
SELECT * FROM empleado;
```

```
mysql> SELECT * FROM empleado;
```

id	nombre
1	Luis Lopez
2	Ana Sanchez
3	Raul Trejo
4	Jorge Campos
5	NULL
6	NULL

```
6 rows in set (0.00 sec)
```

También podemos seleccionar atributos específicos:

```
SELECT id, nombre FROM empleado;
```

```
mysql> SELECT id, nombre FROM empleado;
```

id	nombre
1	Luis Lopez
2	Ana Sanchez
3	Raul Trejo
4	Jorge Campos
5	NULL
6	NULL

```
6 rows in set (0.00 sec)
```

Filtrar y proyectar datos

El comando `SELECT` nos permite aplicar operaciones de selección y proyección de álgebra relacional. Podemos:

- Filtrar filas que cumplan una condición.
- Seleccionar columnas específicas.
- Reordenar columnas.
- Renombrar atributos en los resultados.

Ejemplo

Trabajemos ahora con el ejemplo de los autos. Si queremos recrear la tabla que mostramos al inicio de esta lección:

Número de serie	Marca	Submarca	Modelo	Color	Transmisión	Recorrido
3VWU67FR7RE3	Volkswagen	Polo	2015	Gris Platino	Manual	125,000
HDG6JBSD7FDT	Honda	Civic	2020	Plata Diamante	Automática	40,000
3VWGCISDS7D7	VW	Tiguan	2012	Blanco	Automática	132,000
6TDG675E54GY	Toyota	Corolla	2022	Rojo	Manual	30,000

Podemos insertar los valores con las siguientes instrucciones:

```
INSERT INTO marca VALUES(1,"VOLKSWAGEN"), (2, "HONDA"), (3, "TOYOTA");
```

```
INSERT INTO auto VALUES
("3VWU67FR7RE3", 1, "POLO",      2015, "gris platino", "MANUAL", 125000),
("HDG6JBSD7FDT", 2, "CIVIC",      2020, "PLATA DIAMANTE", "AUTOMÁTICA", 40000),
("3VWGCISDS7D7", 1, "TIGUAN",     2012, "BLANCO",      "AUTOMÁTICA", 132000),
("6TDG675E54GY", 3, "COROLLA",    2022, "ROJO",        "MANUAL",     30000) ;
```

También podemos filtrar valores utilizando comparaciones, por ejemplo, para ver la submarca, el color y el modelo de los autos del 2020 o superiores, podemos usar la siguiente instrucción:

```
SELECT submarca, color, modelo FROM auto WHERE modelo
>= 2020;
```

```
mysql> SELECT submarca, color, modelo FROM auto WHERE modelo >= 2020;
+-----+-----+-----+
| submarca | color          | modelo |
+-----+-----+-----+
| CIVIC    | PLATA DIAMANTE | 2020   |
| COROLLA  | ROJO           | 2022   |
+-----+-----+-----+
2 rows in set (0.01 sec)
```

Además, podemos filtrar información utilizando **conectores lógicos**. Por ejemplo, si quisiéramos ver los autos de las marcas de origen japonés (HONDA y TOYOTA), podríamos hacerlo con el conector **OR**, de la siguiente manera:

```
mysql> SELECT * FROM auto WHERE id_marca = 2 OR id_marca = 3;
+-----+-----+-----+-----+-----+-----+-----+
| vin          | id_marca | submarca | modelo | color          | transmision | recorrido |
+-----+-----+-----+-----+-----+-----+-----+
| HDG6JBSD7FDT | 2        | CIVIC    | 2021   | PLATA DIAMANTE | MANUAL      | 40000    |
| 6TDG675E54GY | 3        | COROLLA  | 2022   | ROJO           | MANUAL      | 30500    |
+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

En el caso de nuestra tabla, también podemos pedir que se nos muestren todos los atributos de los autos cuya marca sea diferente de 1 (por ejemplo, si queremos ver todos los autos que no pertenezcan a VOLKSWAGEN):

```
SELECT * FROM auto WHERE id_marca <> 1;
```

```
mysql> SELECT * FROM auto WHERE id_marca <> 1;
+-----+-----+-----+-----+-----+-----+-----+
| vin          | id_marca | submarca | modelo | color          | transmision | recorrido |
+-----+-----+-----+-----+-----+-----+-----+
| HDG6JBSD7FDT | 2        | CIVIC    | 2021   | PLATA DIAMANTE | MANUAL      | 40000     |
| 6TDG675E54GY | 3        | COROLLA  | 2022   | ROJO           | MANUAL      | 30500     |
+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

En MySQL, también es posible usar el operador `!=` para denotar “diferente”, como en algunos lenguajes de programación:

```
SELECT * FROM auto WHERE id_marca != 1;
```

Para ver más detalles sobre el uso de la instrucción `SELECT`, puedes revisar la documentación oficial de MySQL (recuerda consultar la sección de Material Adicional).

Actualizar datos

Otro tipo de instrucciones de manipulación de datos involucran la actualización de información en una tabla. Esto se hace con el comando `UPDATE`, cuya sintaxis es:

```
UPDATE tabla SET atributo_a_modificar = nuevo_valor WHERE
condicion
```

Este comando comienza con la palabra `UPDATE`, seguida del nombre de la tabla que vamos a actualizar. Luego, usamos `SET` para indicar el atributo cuyo valor queremos modificar, seguido del signo igual (`=`) y el nuevo valor. Finalmente, debemos especificar una condición que los registros deben cumplir para ser actualizados.



Aquí hay que tener mucho cuidado. Si olvidamos escribir la condición, es decir, la parte del `where`, la instrucción va a actualizar todas las filas de la tabla.

Vamos a hacer un par de actualizaciones en el ejemplo de los autos. Recordemos que la tabla tiene la siguiente información:

```
mysql> SELECT * FROM auto;
```

vin	id_marca	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000
HDG6JBSD7FDT	2	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	3	COROLLA	2022	ROJO	MANUAL	30000

```
4 rows in set (0.00 sec)
```

Primero, actualicemos el modelo del CIVIC a 2021, y su transmisión a manual. Para esto, haremos lo siguiente:

```
UPDATE auto SET modelo = 2021, transmisión = "MANUAL"
WHERE vin = "HDG6JBSD7FDT";
```

```
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM auto;
```

vin	id_marca	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000
HDG6JBSD7FDT	2	CIVIC	2021	PLATA DIAMANTE	MANUAL	40000
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	3	COROLLA	2022	ROJO	MANUAL	30000

```
4 rows in set (0.00 sec)
```

Como puedes ver, en esta instrucción `UPDATE` estamos actualizando el valor de más de un atributo (`SET modelo = 2021, transmisión = "MANUAL"`). Un detalle importante es que al final de la consulta se utiliza `WHERE vin = "HDG6JBSD7FDT"`. Podríamos haber hecho este cambio usando, por ejemplo, `WHERE submarca = "CIVIC"`, y en este caso específico, el resultado hubiera sido el mismo, ya que solo tenemos un "CIVIC". Sin embargo, cuando se actualiza un solo registro, es recomendable usar la llave primaria para asegurarnos de que estamos actualizando el registro correcto.

Por otro lado, si quisiéramos modificar todos los registros que cumplan con una cierta condición, podríamos usar cualquier otro atributo o combinación de atributos. Por ejemplo, ahora podríamos actualizar el kilometraje del COROLLA a 30,500:

```
UPDATE auto SET recorrido = 30500 WHERE submarca = "COROLLA" AND modelo = 2022;
```

```
mysql> UPDATE auto SET recorrido = 30500 WHERE submarca = "COROLLA" AND modelo = 2022;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM auto;
```

vin	id_marca	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000
HDG6JBSD7FDT	2	CIVIC	2021	PLATA DIAMANTE	MANUAL	40000
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	3	COROLLA	2022	ROJO	MANUAL	30500

```
4 rows in set (0.00 sec)
```

Aquí utilizamos el conector `AND` para ejemplificar qué podríamos hacer si tuviéramos 2 autos "COROLLA".

Eliminar datos

Para borrar datos de una tabla, podemos utilizar el comando `DELETE`:

```
DELETE FROM tabla WHERE condición
```



Aquí también hay que tener mucho cuidado, pues la sintaxis es muy similar a la de `UPDATE`: si omitimos la parte del `WHERE`, estaríamos eliminando todos los registros de nuestra tabla.

Veamos a continuación algunos ejemplos. Supongamos que por algún motivo queremos eliminar el registro del COROLLA. La instrucción quedaría así:

```
DELETE FROM auto WHERE submarca = "COROLLA";
```

```
mysql> DELETE FROM auto WHERE submarca = "COROLLA";
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM auto;
+-----+-----+-----+-----+-----+-----+-----+
| vin          | id_marca | submarca | modelo | color          | transmision | recorrido |
+-----+-----+-----+-----+-----+-----+-----+
| 3VWU67FR7RE3 | 1        | POLO     | 2015   | gris platino   | MANUAL      | 125000    |
| HDG6JBSD7FDT | 2        | CIVIC    | 2021   | PLATA DIAMANTE | MANUAL      | 40000     |
| 3VWGCISDS7D7 | 1        | TIGUAN   | 2012   | BLANCO         | AUTOMATICA  | 132000    |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Un error común al ejecutar este tipo de instrucciones es que, al copiar y pegar comandos desde algunos documentos, el caracter de comillas (") no se copia correctamente o no coincide con el que espera el manejador de bases de datos (MySQL).

En algunas ocasiones, puede ser útil eliminar todos los registros de una tabla. Esto se puede hacer con la instrucción `DELETE FROM tabla`, o también se puede usar el comando `TRUNCATE tabla`.

La instrucción `TRUNCATE` elimina todos los registros de la tabla, pero mantiene su estructura. En cambio, la instrucción `DROP TABLE` elimina completamente la tabla, incluyendo todos los registros que pueda contener.

Por ejemplo, para borrar todos los registros de nuestra tabla de autos, podemos ejecutar la siguiente instrucción:

```
TRUNCATE auto;
```

```
mysql> TRUNCATE auto;
Query OK, 0 rows affected (0.09 sec)

mysql> SELECT * FROM auto;
Empty set (0.00 sec)
```

Recuperación de datos con `SELECT` y `JOIN` para combinar datos de múltiples tablas

En los ejemplos anteriores, hemos trabajado únicamente con la tabla de autos y hemos dejado de lado la tabla de marcas. Esto se debe a que aún no hemos explorado el uso del operador `JOIN`, el cual nos permite combinar datos de múltiples tablas relacionadas. En nuestro caso, las marcas de autos y los automóviles están almacenados en tablas separadas.

El operador `JOIN` en SQL se utiliza para combinar filas de dos o más tablas que comparten una relación mediante una condición específica. Por ejemplo, en nuestra base de datos, los automóviles de la marca HONDA tienen el identificador de marca (`id_marca`) igual a 2. Esto significa que, en la tabla de marcas, el registro con `id_marca = 2` corresponde a HONDA.

Nuestro objetivo es recuperar, restaurar o volver a construir una tabla que muestre que el CIVIC es un auto de la marca HONDA, que el auto POLO es de la marca VOLKSWAGEN y que el TIGUAN es de la marca VOLKSWAGEN.

Existen varias formas de relacionar nuestras tablas de autos y marcas. Antes de explorarlas, primero revisemos cómo están estructuradas nuestras tablas:

```
mysql> SELECT * FROM auto;
```

vin	id_marca	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000
HDG6JBSD7FDT	2	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	3	COROLLA	2022	ROJO	MANUAL	30000

```
4 rows in set (0.00 sec)
```



```
mysql> SELECT * FROM marca;
```

id_marca	nombre_marca
1	VOLKSWAGEN
2	HONDA
3	TOYOTA

```
3 rows in set (0.00 sec)
```

La primera, que es una forma muy simple y probablemente no muy eficiente, es hacer un producto de todos los autos contra todas las marcas, y luego filtrar solamente las filas en las que coincida el identificador de la marca en la tabla de autos con el identificador de la marca en la tabla de marcas.

Para hacer un producto entre dos tablas, podemos ejecutar una operación de `SELECT FROM` en ambas tablas:

```
SELECT * FROM auto, marca;
```

```
mysql> SELECT * FROM auto, marca;
```

vin	id_marca	submarca	modelo	color	transmision	recorrido	id_marca	nombre_marca
3VWU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000	3	TOYOTA
3VWU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000	2	HONDA
3VWU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000	1	VOLKSWAGEN
HDG6JBSD7FDT	2	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000	3	TOYOTA
HDG6JBSD7FDT	2	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000	2	HONDA
HDG6JBSD7FDT	2	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000	1	VOLKSWAGEN
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000	3	TOYOTA
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000	2	HONDA
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000	1	VOLKSWAGEN
6TDG675E54GY	3	COROLLA	2022	ROJO	MANUAL	30000	3	TOYOTA
6TDG675E54GY	3	COROLLA	2022	ROJO	MANUAL	30000	2	HONDA
6TDG675E54GY	3	COROLLA	2022	ROJO	MANUAL	30000	1	VOLKSWAGEN

12 rows in set (0.00 sec)

Sin embargo, esto no es lo que queremos, ya que esta consulta genera combinaciones incorrectas y relaciona todas las filas de la tabla `auto` con todas las filas de la tabla `marca`. Esto podría dar resultados erróneos, como que el auto POLO pertenezca a TOYOTA, HONDA y VOLKSWAGEN al mismo tiempo.

Para obtener únicamente las combinaciones correctas, debemos agregar una condición que compare el identificador de la marca en ambas tablas, como la siguiente:

```
SELECT * FROM auto, marca WHERE auto.id_marca =  
marca.id_marca;
```

```
mysql> SELECT * FROM auto, marca WHERE auto.id_marca = marca.id_marca;
```

vin	id_marca	submarca	modelo	color	transmision	recorrido	id_marca	nombre_marca
3VWU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000	1	VOLKSWAGEN
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000	1	VOLKSWAGEN
HDG6JBSD7FDT	2	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000	2	HONDA
6TDG675E54GY	3	COROLLA	2022	ROJO	MANUAL	30000	3	TOYOTA

4 rows in set (0.00 sec)

Una forma más eficiente de realizar esta relación es utilizando el operador `JOIN`, el cual nos permite especificar directamente las columnas que deben coincidir en ambas tablas.

La sintaxis general del operador `JOIN` es:

```
SELECT * FROM tabla1 JOIN tabla2 ON tabla1.atributo =
tabla2.atributo;
```

En nuestro ejemplo de los autos, la instrucción quedaría de la siguiente manera:

```
SELECT * FROM auto JOIN marca ON auto.id_marca =
marca.id_marca;
```

```
mysql> SELECT * FROM auto JOIN marca ON auto.id_marca = marca.id_marca;
```

vin	id_marca	submarca	modelo	color	transmision	recorrido	id_marca	nombre_marca
3VMU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000	1	VOLKSWAGEN
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000	1	VOLKSWAGEN
HDG6JBSD7FDT	2	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000	2	HONDA
6TDG675E54GY	3	COROLLA	2022	ROJO	MANUAL	30000	3	TOYOTA

```
4 rows in set (0.00 sec)
```

Si las tablas que estamos relacionando tienen columnas con el mismo nombre y estas representan la clave de relación entre ambas, podemos utilizar el operador `NATURAL JOIN`. Este operador busca automáticamente los atributos comunes en ambas tablas y los usa para hacer la unión sin necesidad de especificar la condición manualmente.

La sintaxis del operador `NATURAL JOIN` es:

```
SELECT * FROM tabla1 NATURAL JOIN tabla2;
```

Aplicándolo a nuestro caso, la consulta quedaría de la siguiente manera:

```
SELECT * FROM auto NATURAL JOIN marca;
```

```
mysql> SELECT * FROM auto NATURAL JOIN marca;
```

id_marca	vin	submarca	modelo	color	transmision	recorrido	nombre_marca
1	3VWU67FR7RE3	POLO	2015	gris platino	MANUAL	125000	VOLKSWAGEN
1	3VWGCISDS7D7	TIGUAN	2012	BLANCO	AUTOMATICA	132000	VOLKSWAGEN
2	HDG6JBSD7FDT	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000	HONDA
3	6TDG675E54GY	COROLLA	2022	ROJO	MANUAL	30000	TOYOTA

4 rows in set (0.00 sec)

Ahora, vamos a modificar los valores en nuestras tablas; agregaremos más datos para poder ejemplificar otras formas de utilizar el operador JOIN. Observa la siguiente instrucción:

```
INSERT INTO marca VALUES (4,"FORD"), (5, "MAZDA");
```

```
INSERT INTO auto VALUES  
("3VWU67FR7RE3", NULL, "VITARA", 2016, "BLANCO", "AUTOMATICA", 213000),  
("3VWGCISDS7D7", NULL, "WRX", 2020, "BLANCO", "MANUAL", 50000);
```

De esta manera, nuestras tablas ahora contienen los siguientes registros:

```
mysql> SELECT * FROM marca;
```

id_marca	nombre_marca
1	VOLKSWAGEN
2	HONDA
3	TOYOTA
4	FORD
5	MAZDA

5 rows in set (0.00 sec)

```
mysql> SELECT * FROM auto;
```

vin	id_marca	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000
HDG6JBSD7FDT	2	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	3	COROLLA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	NULL	VITARA	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	NULL	WRX	2020	BLANCO	MANUAL	50000

6 rows in set (0.00 sec)

Como puedes ver, añadimos autos que no están relacionados con una marca, así como marcas que no están relacionadas con ningún auto. Este es un escenario común que nos permitirá ejemplificar el uso de `LEFT JOIN`, `RIGHT JOIN` y `FULL OUTER JOIN`.

Hasta ahora, la operación `JOIN` que realizamos solo nos mostró los autos con una marca correspondiente, es decir, todos los registros coincidían perfectamente. Sin embargo, ahora que hemos incorporado autos sin marca y marcas sin autos, veamos qué sucede al ejecutar la operación `JOIN` en este nuevo contexto:

```
SELECT * FROM auto NATURAL JOIN marca;
```

```
mysql> SELECT * FROM auto NATURAL JOIN marca;
```

id_marca	vin	submarca	modelo	color	transmision	recorrido	nombre_marca
1	3VWU67FR7RE3	POLO	2015	gris platino	MANUAL	125000	VOLKSWAGEN
1	3VWGCISDS7D7	TIGUAN	2012	BLANCO	AUTOMATICA	132000	VOLKSWAGEN
2	HDG6JBSD7FDT	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000	HONDA
3	6TDG675E54GY	COROLLA	2022	ROJO	MANUAL	30000	TOYOTA

4 rows in set (0.00 sec)

Y ahora observa lo que sucede con la siguiente instrucción:

```
SELECT * FROM auto INNER JOIN marca ON auto.id_marca =  
marca.id_marca;
```

```
mysql> SELECT * FROM auto INNER JOIN marca ON auto.id_marca =  
marca.id_marca;
```

vin	id_marca	submarca	modelo	color	transmision	recorrido	id_marca	nombre_marca
3VWU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000	1	VOLKSWAGEN
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000	1	VOLKSWAGEN
HDG6JBSD7FDT	2	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000	2	HONDA
6TDG675E54GY	3	COROLLA	2022	ROJO	MANUAL	30000	3	TOYOTA

4 rows in set (0.00 sec)

Como puedes ver, el resultado es el mismo, ya que la operación de `JOIN` es, por *default*, un `INNER JOIN`.

También podemos especificar cuáles son las columnas que queremos desplegar, de este modo:

```
SELECT vin, nombre_marca, submarca, modelo, color,  
transmision, recorrido  
FROM auto NATURAL JOIN marca;
```

```
mysql> SELECT vin, nombre_marca, submarca, modelo, color, transmision, recorrido FROM auto NATURAL JOIN marca;
```

vin	nombre_marca	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	VOLKSWAGEN	POLO	2015	gris platino	MANUAL	125000
3VWGCISDS7D7	VOLKSWAGEN	TIGUAN	2012	BLANCO	AUTOMATICA	132000
HDG6JBSD7FDT	HONDA	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000
6TDG675E54GY	TOYOTA	COROLLA	2022	ROJO	MANUAL	30000

```
4 rows in set (0.00 sec)
```

Pero ¿qué pasaría si, por ejemplo, quisiéramos mostrar todos los autos, aunque no estén relacionados con una marca? Aquí es donde entra el operador `LEFT JOIN`, el cual incluye todos los elementos del lado izquierdo.

Cuando escribimos una operación `JOIN`, escribimos una tabla `JOIN` otra tabla. La tabla que aparece a la izquierda es la que incluirá todos sus registros en el resultado, incluso si no tienen una relación con la otra tabla.

Por ejemplo, si ejecutamos:

```
autos LEFT JOIN marca
```

Todas las filas de la tabla `autos` aparecerán en la respuesta (ya que está a la izquierda), incluso si algún auto no tiene una marca asociada.

Observa la siguiente instrucción:

```
SELECT vin, nombre_marca, submarca, modelo, color,
transmision, recorrido FROM auto LEFT JOIN marca ON
auto.id_marca = marca.id_marca;
```

Al ejecutarla, obtendremos esto:

```
mysql> SELECT * FROM auto LEFT JOIN marca ON auto.id_marca = marca.id_marca;
```

vin	id_marca	submarca	modelo	color	transmision	recorrido	id_marca	nombre_marca
3VMU67FR7RE3	1	POLO	2015	gris platino	MANUAL	125000	1	VOLKSWAGEN
HDG6JBSD7FDT	2	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000	2	HONDA
3VWGCISDS7D7	1	TIGUAN	2012	BLANCO	AUTOMATICA	132000	1	VOLKSWAGEN
6TDG675E54G4	3	COROLLA	2022	ROJO	MANUAL	30000	3	TOYOTA
JD76F6SDF7DF	NULL	VITARA	2016	BLANCO	AUTOMATICA	213000	NULL	NULL
9DTF6S5SG5G5	NULL	WRX	2020	BLANCO	MANUAL	50000	NULL	NULL

6 rows in set (0.00 sec)

Ahora, observa lo siguiente:

```
SELECT
    vin,
    nombre_marca,
    submarca,
    modelo,
    color,
    transmision,
    recorrido
FROM auto LEFT JOIN marca
    ON auto.id_marca = marca.id_marca;
```

Aquí, la consulta se separó en varias líneas para tener una mejor lectura. Al ejecutarla, obtendremos este resultado:

```
mysql> SELECT
->   vin,
->   nombre_marca,
->   submarca,
->   modelo,
->   color,
->   transmision,
->   recorrido
-> FROM auto LEFT JOIN marca
->   ON auto.id_marca = marca.id_marca;
```

vin	nombre_marca	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	VOLKSWAGEN	POLO	2015	gris platino	MANUAL	125000
HGG6JBSD7FDT	HONDA	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	VOLKSWAGEN	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	TOYOTA	COROLLA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	NULL	VITARA	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	NULL	WRX	2020	BLANCO	MANUAL	50000

6 rows in set (0.00 sec)

Como puedes observar, al no estar relacionados con una marca, los autos VITARA y WRX aparecen con el valor `NULL` en el atributo de marca.

Ahora, ¿qué sucede si queremos ver todas las marcas, incluso aquellas que no tienen autos asociados? En este caso, utilizamos el `RIGHT JOIN`. Este tipo de `JOIN` asegura que todos los registros de la tabla a la derecha se incluyan en el resultado, sin importar si tienen una relación con la tabla de la izquierda.

Por ejemplo, al ejecutar:

```
autos LEFT JOIN marca
```

Todas las marcas aparecerán en el resultado, incluso si no hay autos registrados para alguna de ellas.

Veamos el ejemplo con los autos:

```
SELECT
    vin,
    nombre_marca,
    submarca,
    modelo,
    color,
    transmision,
    recorrido
FROM auto RIGHT JOIN marca
    ON auto.id_marca = marca.id_marca;
```

Observa el resultado a continuación:

```
mysql> SELECT
->     vin,
->     nombre_marca,
->     submarca,
->     modelo,
->     color,
->     transmision,
->     recorrido
-> FROM auto RIGHT JOIN marca
->     ON auto.id_marca = marca.id_marca;
```

vin	nombre_marca	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	VOLKSWAGEN	POLO	2015	gris platino	MANUAL	125000
3VWGCISDS7D7	VOLKSWAGEN	TIGUAN	2012	BLANCO	AUTOMATICA	132000
HDG6JBSD7FDT	HONDA	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000
6TDG675E54GY	TOYOTA	COROLLA	2022	ROJO	MANUAL	30000
NULL	FORD	NULL	NULL	NULL	NULL	NULL
NULL	MAZDA	NULL	NULL	NULL	NULL	NULL

6 rows in set (0.00 sec)

Después de ver `LEFT JOIN` y `RIGHT JOIN`, es natural preguntarnos si existe una forma de incluir todos los registros de ambas tablas, independientemente de si tienen una relación. Para esto, usamos `FULL OUTER JOIN`. Sin embargo, algunos sistemas gestores de bases de datos, como MySQL y MariaDB, no lo tienen implementado de forma nativa. Afortunadamente, podemos simularlo utilizando la operación `UNION`.

Recordemos que la operación UNION combina los resultados de dos consultas. Entonces, si realizamos un LEFT JOIN y un RIGHT JOIN, y luego aplicamos UNION, obtendremos el mismo resultado que un FULL OUTER JOIN.

Aquí te ponemos el ejemplo con las tablas de los autos:

```
SELECT
    vin,
    nombre_marca,
    submarca,
    modelo,
    color,
    transmision,
    recorrido
FROM auto LEFT JOIN marca
    ON auto.id_marca = marca.id_marca
```

UNION

```
SELECT
    vin,
    nombre_marca,
    submarca,
    modelo,
    color,
    transmision,
    recorrido
FROM auto RIGHT JOIN marca
    ON auto.id_marca = marca.id_marca;
```

Al ejecutar lo anterior, obtendremos esto:

```
mysql> SELECT
->   vin,
->   nombre_marca,
->   submarca,
->   modelo,
->   color,
->   transmision,
->   recorrido
-> FROM auto LEFT JOIN marca
->   ON auto.id_marca = marca.id_marca
->
-> UNION
->
-> SELECT
->   vin,
->   nombre_marca,
->   submarca,
->   modelo,
->   color,
->   transmision,
->   recorrido
-> FROM auto RIGHT JOIN marca
->   ON auto.id_marca = marca.id_marca;
```

vin	nombre_marca	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	VOLKSWAGEN	POLO	2015	gris platino	MANUAL	125000
HDG6JBSD7FDT	HONDA	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	VOLKSWAGEN	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	TOYOTA	COROLLA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	NULL	VITARA	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	NULL	WRX	2020	BLANCO	MANUAL	50000
NULL	FORD	NULL	NULL	NULL	NULL	NULL
NULL	MAZDA	NULL	NULL	NULL	NULL	NULL

8 rows in set (0.00 sec)

Aquí hay una versión un poco más compacta de la misma operación:

```
SELECT vin, nombre_marca, submarca, modelo, color,
transmision, recorrido
FROM auto LEFT JOIN marca
  ON auto.id_marca = marca.id_marca
```

UNION

```
SELECT vin, nombre_marca, submarca, modelo, color,
transmision, recorrido
FROM auto RIGHT JOIN marca
ON auto.id_marca = marca.id_marca;
```

Este es el resultado:

```
mysql> SELECT vin, nombre_marca, submarca, modelo, color, transmision, recorrido
-> FROM auto LEFT JOIN marca
-> ON auto.id_marca = marca.id_marca
-> UNION
-> SELECT vin, nombre_marca, submarca, modelo, color, transmision, recorrido
-> FROM auto RIGHT JOIN marca
-> ON auto.id_marca = marca.id_marca;
```

vin	nombre_marca	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	VOLKSWAGEN	POLO	2015	gris platino	MANUAL	125000
HDG6JBSD7FDT	HONDA	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	VOLKSWAGEN	TIGUAN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	TOYOTA	COROLLA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	NULL	VITARA	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	NULL	WRX	2020	BLANCO	MANUAL	50000
NULL	FORD	NULL	NULL	NULL	NULL	NULL
NULL	MAZDA	NULL	NULL	NULL	NULL	NULL

8 rows in set (0.00 sec)

Como puedes ver, aparecen tanto los autos que no tienen una marca asociada, como las marcas que no tienen autos asociados.

Concepto de normalización y su importancia en bases de datos

En los ejemplos anteriores, separamos la información de los autos en dos tablas: una para los autos y otra para las marcas. Esta separación nos permitió ejemplificar el uso de llaves primarias y llaves foráneas, aunque aún existen oportunidades para mejorar la estructura de los datos.

Si almacenáramos toda la información en una sola tabla, podríamos registrar el nombre de una marca de diferentes formas, lo que generaría errores al recuperar la información. Sin embargo, al dividir los datos en una tabla de marcas y otra de autos, y al registrar un auto seleccionando una marca previamente almacenada, reducimos significativamente estos errores.

La **normalización** es un proceso que optimiza la estructura de la base de datos para eliminar redundancias y mejorar la integridad de los datos. Este proceso se aplica después del diseño inicial, donde primero identificamos las entidades (qué vamos a almacenar), sus atributos (qué características de cada entidad guardaremos) y sus relaciones (cómo interactúan entre sí). A menudo, estas relaciones se representan mediante tablas vinculadas a través de llaves primarias y foráneas.

Por ejemplo, sin normalización, podríamos registrar erróneamente un CIVIC como un auto de la marca VOLKSWAGEN, lo cual es incorrecto. Las restricciones de una base de datos no son reglas predefinidas, sino que deben reflejar la realidad del modelo que estamos construyendo. Más adelante, exploraremos cómo organizar los datos en diferentes tablas para garantizar que se cumplan estas restricciones.

La normalización sigue una serie de pasos conocidos como **formas normales**. Se recomienda alcanzar al menos la tercera forma normal para que nuestra base de datos sea robusta y eficiente.

Primera forma normal

La primera forma normal nos dice que en cada intersección de fila y columna debe contener un único valor atómico, es decir, no se pueden almacenar listas o múltiples valores en una misma celda.

Por ejemplo, si queremos registrar números telefónicos de personas, y algunas tienen más de un número (personal y de trabajo), no debemos almacenarlos en una sola celda. En su lugar, creamos una tabla adicional con el identificador de la persona y un número por fila, incluyendo un atributo que especifique si es un teléfono personal o laboral.

Segunda forma normal

La segunda forma normal está muy relacionada con el concepto de **dependencia funcional**. Cuando creamos una tabla, podemos trabajar con llaves primarias para identificar de manera única un registro en esa tabla. Esta llave puede estar compuesta de varios atributos o de uno solo. Si nuestra llave primaria es atómica, es decir, si es de un solo atributo, entonces nuestra tabla ya está en segunda forma normal.

Pero ¿qué pasa cuando tenemos la combinación de más de un atributo como llave primaria? En este caso, todos los demás atributos de la tabla deben depender completamente de la combinación de valores que forman la llave primaria. Si algún atributo depende solo de una parte de la llave primaria y no de su totalidad, significa que la tabla **no cumple** con la segunda forma normal. Para corregirlo, es necesario trasladar ese atributo a una nueva tabla junto con el atributo del que depende, asegurando así una mejor organización y evitando redundancias en la base de datos.

Tercera forma normal

La tercera forma normal tiene que ver con el concepto de **dependencia transitiva**, es decir, cuando un atributo depende de otro que, a su vez, depende de la llave primaria.

En nuestro ejemplo de los autos, tenemos una tabla con los atributos número de serie, submarca y marca de un auto:

- La submarca depende del número de serie.
- La marca depende de la submarca (ejemplo: todos los CIVIC son de HONDA).

Aquí hay una dependencia transitiva, lo que puede generar inconsistencias. Para evitarlo, debemos crear una nueva tabla donde se relacionen marca y submarca, eliminando así la redundancia y asegurando que un CIVIC no pueda registrarse como parte de TOYOTA o un COROLLA como parte de NISSAN.

A continuación, presentamos un ejemplo de la definición de las tablas `auto`, `marca` y `submarca`, donde se ha eliminado la dependencia transitiva. También mostramos ejemplos de consultas con `NATURAL JOIN` y `LEFT JOIN`:

```
CREATE TABLE marca(  
  id_marca SMALLINT NOT NULL,  
  marca VARCHAR(30) NOT NULL,  
  PRIMARY KEY(id_marca)  
);  
  
CREATE TABLE submarca(  
  submarca VARCHAR(30) NOT NULL,  
  id_marca SMALLINT,  
  PRIMARY KEY(submarca),  
  FOREIGN KEY(id_marca) REFERENCES marca(id_marca)  
    ON DELETE RESTRICT  
    ON UPDATE CASCADE  
);
```

```

CREATE TABLE auto (
  vin VARCHAR(17) ,
  submarca VARCHAR(30),
  modelo YEAR,
  color VARCHAR(30),
  transmision ENUM("MANUAL","AUTOMATICA"),
  recorrido int,
  FOREIGN KEY(submarca) REFERENCES submarca(submarca)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);

INSERT INTO marca VALUES(1,"VOLKSWAGEN"), (2, "HONDA"), (3, "TOYOTA");
INSERT INTO marca VALUES(4,"FORD"), (5, "MAZDA");

INSERT INTO submarca VALUES("POLO",1);
INSERT INTO submarca VALUES("CIVIC",2);
INSERT INTO submarca VALUES("TIGUAN",1);
INSERT INTO submarca VALUES("COROLLA",3);
INSERT INTO submarca VALUES("VITARA",NULL);
INSERT INTO submarca VALUES("WRX",NULL);

INSERT INTO auto VALUES
  ("3VWU67FR7RE3","POLO", 2015, "GRIS PLATINO", "MANUAL", 125000),
  ("HDG6JBSD7FDT","CIVIC", 2020, "PLATA DIAMANTE", "AUTOMÁTICA", 40000),
  ("3VWGCISDS7D7","TIGUAN", 2012, "BLANCO", "AUTOMÁTICA", 132000),
  ("6TDG675E54GY","COROLLA", 2022, "ROJO", "MANUAL", 30000) ;

INSERT INTO auto VALUES
  ("JD76F6SDF7DF", "VITARA", 2016, "BLANCO", "AUTOMATICA", 213000),
  ("9DTF6S5SG5G5","WRX", 2020, "BLANCO", "MANUAL", 50000);

select vin, marca, submarca, modelo, color, transmision, recorrido from
auto NATURAL JOIN submarca NATURAL JOIN marca;

select vin, auto.submarca, marca, modelo, color, transmision, recorri-
do from auto LEFT JOIN submarca on auto.submarca = submarca.submarca LEFT
JOIN marca on submarca.id_marca = marca.id_marca;

```

```
mysql> SELECT vin, marca, submarca, modelo, color, transmision, recorrido
-> FROM auto NATURAL JOIN submarca NATURAL JOIN marca;
```

vin	marca	submarca	modelo	color	transmision	recorrido
3VWU67FR7RE3	VOLKSWAGEN	POLO	2015	GRIS PLATINO	MANUAL	125000
3VWGCISDS7D7	VOLKSWAGEN	TIGUAN	2012	BLANCO	AUTOMATICA	132000
HDG6JBSD7FDT	HONDA	CIVIC	2020	PLATA DIAMANTE	AUTOMATICA	40000
6TDG675E54GY	TOYOTA	COROLLA	2022	ROJO	MANUAL	30000

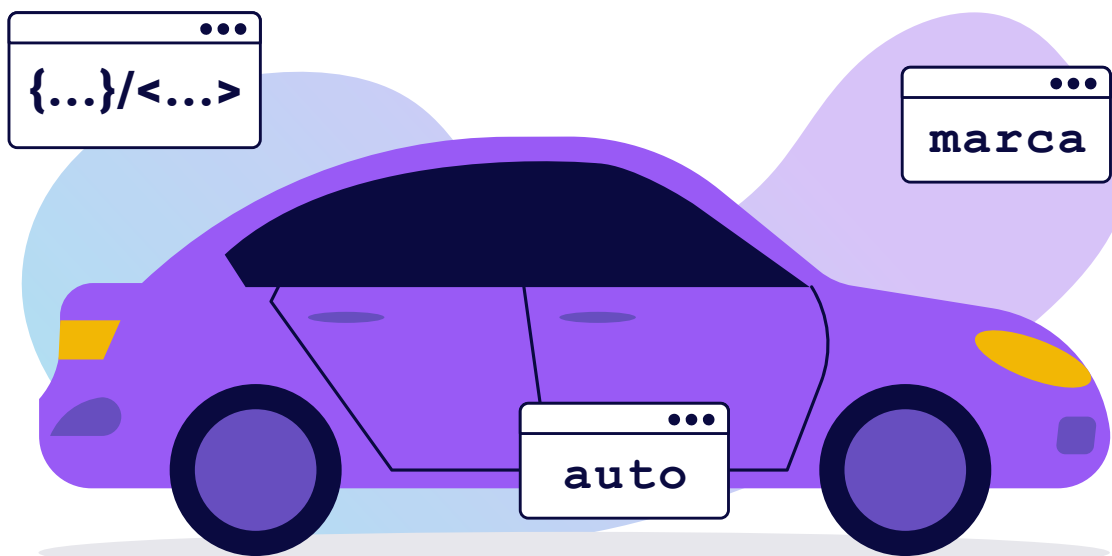
```
4 rows in set (0.00 sec)
```

```
mysql> SELECT vin, auto.submarca, marca, modelo, color, transmision, recorrido
-> FROM auto LEFT JOIN submarca ON auto.submarca = submarca.submarca
-> LEFT JOIN marca ON submarca.id_marca = marca.id_marca;
```

vin	submarca	marca	modelo	color	transmision	recorrido
3VWU67FR7RE3	POLO	VOLKSWAGEN	2015	GRIS PLATINO	MANUAL	125000
HDG6JBSD7FDT	CIVIC	HONDA	2020	PLATA DIAMANTE	AUTOMATICA	40000
3VWGCISDS7D7	TIGUAN	VOLKSWAGEN	2012	BLANCO	AUTOMATICA	132000
6TDG675E54GY	COROLLA	TOYOTA	2022	ROJO	MANUAL	30000
JD76F6SDF7DF	VITARA	NULL	2016	BLANCO	AUTOMATICA	213000
9DTF6S5SG5G5	WRX	NULL	2020	BLANCO	MANUAL	50000

```
6 rows in set (0.00 sec)
```

Si bien existen más formas normales, en la mayoría de los casos, alcanzar la tercera forma normal es suficiente para garantizar una base de datos bien estructurada y eficiente.



| Para terminar...

Para cerrar esta lección, hemos explorado los fundamentos de SQL, desde la creación de esquemas de bases de datos hasta las diferentes instrucciones para la manipulación y recuperación de información. También analizamos la importancia de la normalización para optimizar la estructura de una base de datos, reduciendo redundancias y mejorando su integridad.

Ahora, te invitamos a reflexionar: ¿cómo crees que pueda influir un buen diseño de base de datos en el rendimiento y mantenimiento del sistema?, ¿qué ventajas crees que nos pueda ofrecer la normalización frente a un diseño sin normalizar?, ¿cómo aplicarías los conocimientos adquiridos hasta el momento para proponer un buen modelo de datos en un proyecto que pueda atender a una necesidad tuya?

Poner en práctica estos conceptos te permitirá desarrollar bases de datos más eficientes, organizadas y escalables.

Sin embargo, esto no termina aún. Continúa con el siguiente recurso, donde pondrás a prueba los conocimientos adquiridos en esta lección.



Glosario

- **Base de datos relacional:** Modelo de base de datos que organiza la información en tablas relacionadas entre sí mediante llaves primarias y foráneas.
- **Dependencia funcional:** Relación en la que el valor de un atributo determina de manera única el valor de otro atributo en la misma tabla.
- **Dependencia transitiva:** Ocurre cuando un atributo depende de otro que, a su vez, depende de la llave primaria, lo que puede generar redundancias.
- **Llave primaria:** Atributo o conjunto de atributos que identifican de manera única cada registro en una tabla.
- **Llave foránea:** Atributo en una tabla que establece una relación con la llave primaria de otra tabla.
- **Normalización:** Proceso que reduce la redundancia y mejora la organización de los datos, dividiéndolos en múltiples tablas relacionadas.

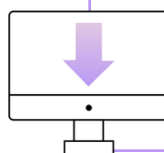
Material adicional

- Tipos de datos en MySQL:
Oracle. (s.f.). *Chapter 13 Data Types* [Capítulo 13 Tipos de datos]. Recuperado el 31 de marzo de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/data-types.html>
- Cómo crear tablas en MySQL:
Oracle. (s.f.). *CREATE TABLE Statement* [Declaración CREATE TABLE]. Recuperado el 31 de marzo de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/create-table.html>
- Uso de la instrucción `SELECT`
Oracle. (s.f.). *SELECT Statement* [Declaración SELECT]. Recuperado el 31 de marzo de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/select.html>

Nota: Los enlaces a las páginas web incluidos en esta sección fueron consultados y verificados por última vez el 31 de marzo de 2025 y su disponibilidad depende del autor.

Bibliografía

Elmasri, R. y Navathe, S. B. (2007). *Fundamentos de sistemas de Bases de Datos (5ta edición)*. Pearson.



Descarga este documento en la sección **"Archivos adjuntos"** de la plataforma.